

Git Commit Signing

Or how I got away with crashing prod after being laid off

Scene 1

Act 1

- Outage Over
 - Company Lost
 - Money
 - Reputation
 - SLAs
- Investigation reveals a malicious commit
 - eg: logic bomb
- Git Commit author is **you**
 - You never wrote that code

Act 2

Managers

Who fires on the spot?

Devs

How do you defend yourself?

How do you prove Git is *wrong* ?

ToC

1. What's my issue with git?
2. Why Should I Care?
3. Mitigation: Git Commit Signing
4. Why Git Signing Saved Me
5. Project-Level Mitigations
6. Closing thoughts

\$ whoami

- Gabriel Fournier
- 15 years DevSecOps
- Sign All My Commits Since ~2020
- Laid off in 2023
 - Crashed Prod on the day I was laid off
 - Accidentally!

Disclaimers

I am not a lawyer

I am not your lawyer

This is Not Legal Advice

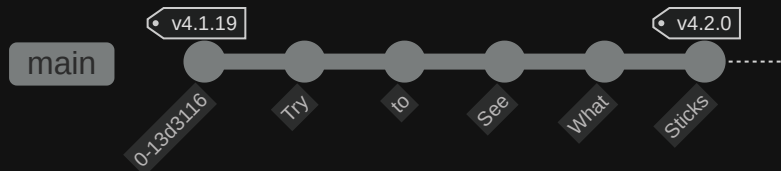
Disclaimers

Git Impersonation Attacks are shown as an educational aid only.
Please replicate demos **responsibly**

What's my issue with git?

Git Push AuthN $\neq$ Git AuthN

Moonlight project

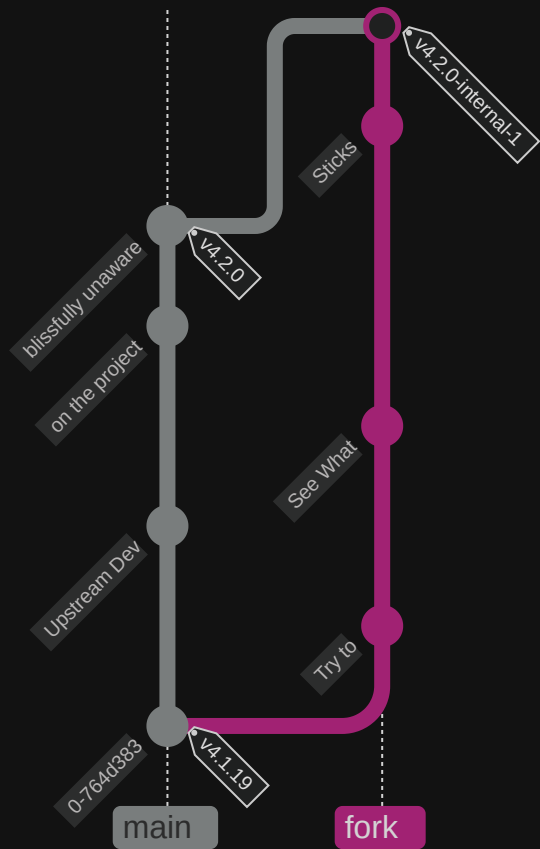


Commit:

- Authors: 1
- Committers: 1
- Pushers: 1

It all checks out, right?

Forked Open Source Project



On My Project:

- Authors: $\infty + 1$
- Committers: $\infty + 1$
- Pushers: 1

Suddenly, you're pushing someone else's commits!

So I can push someone else's commits...

Surely, I Can't commit as someone else?

```
$ docker run -it --rm -v $(pwd):/demo -w /demo alpine sh
```

```
# apk add git
# git config user.name "Gilles Bélanger"
# git config user.email "Gilles.Belanger.0RF0@assnat.qc.ca"
# echo "X50!P%@AP[4\PZX54(P^)7CC)7}\$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!\$H+H*" > eicar

# chmod +x eicar
# git add eicar
# git commit -m "Malicious Commit"
# exit
```

```
$ git log --format=oneline
```

```
$ git push
```

Want to retroactively blame someone else?
Check out Jay Phelps' git-blame-someone-else!

<https://github.com/jayphelps/git-blame-someone-else>

Why Should I Care?

Hanlon's Razor

Never attribute to malice that which is adequately explained by stupidity

Mitigation: Least Privileges



I can't impersonate you on a repo I don't have write access to

Mitigation: Git Commit Signing

Cryptographic Signatures

Triple Guarantees

- Integrity
- Authentication
- Non-Repudiation



Source: Bank of Canada

Generating a Key

```
$ gpg --full-generate-key
```

Answer all the prompts

```
pub   ed25519 2025-10-12 [SC]  
      9D38D37552251C5D354F5FC5A04163ECF659F045  
uid           Gabriel Fournier <gab@demo.dod>
```

Configuring Git to sign commits

```
git config --global user.signingKey 9D38D37552251C5D354F5FC5A04163ECF659F045
git config --global commit.gpgsign true
git config --global tag.gpgsign true
git config --global push.signing if-asked
```

Publish your Public Key to github

```
$ gpg --armor --export 9D38D37552251C5D354F5FC5A04163ECF659F045
```

```
-----BEGIN PGP PUBLIC KEY BLOCK-----
```

```
mDMEaOvShxYJKwYBBAHaRw8BAQdAClN956LDwVfUUeukMiSY72YBDYtgJaY1cLP7  
vnpdney0H0dhYnJpZWwgRm91cm5pZXIgaGdhYkZlZlVlZlRvZD6ImQQTFgoAQRyh  
BJ0403VSJRxdNU9fxaBBY+z2WfBFBQJo69KHAhsDBQkFo5qABQsJCAcCAiICBhUK  
CQgLAgQWAgMBAh4HAheAAAJEKBBY+z2WfBFABQBAIvzJjB6EiMfwHctwXzEmLP  
ril30ZFjew8ZchIF2yD7AQDp2FXzrbnSN8R5tShWJiy+30ETH6tyZLkPWncP3AZ+  
CA==
```

```
=feSD
```

```
-----END PGP PUBLIC KEY BLOCK-----
```

Protip: Private Key Propagation

Do not export your keys, having 1+ key/device is very fine

Enable Vigilant Mode

Default

Vigilant

Verified

Verified

Verified

Partially Verified

Unverified

Unverified

(Nothing)

Unverified

Incident Summary (How I Crashed Prod)

- Wednesday Morning
 - Laid off at the end of the week
- Severance Agreement
 - Conditions include no sabotage

```
1 gab:  
2   user.absent:  
3     purge: true
```

- Salt Apply

- CD starts failing all its jobs
 - No CD is not using my account
- Most every service gets in degraded state
 - Services don't restart

- SaltStack Updated Docker from 19 to 20
 - A config key was changed
 - Sentry didn't restart
 - Our services wait for Sentry when starting

- Update Docker Config
 - Sentry Restarts
 - All Services Bootstrap

Why Git Signing Saved Me

Courts hold Crypto Signatures as Valid

Expose malice from

```
gab:
- user.present:
-   - fullname: Gabriel Fournier
-   - shell: /bin/bash
-   - home: /home/gab
-   - uid: 4000
-   - gid: 4000
-   - groups:
-       - wheel
+ user.absent:
+   purge: true
```

Find a shortcoming in GPG
that invalidates the signature

Please Disclose Responsibly!

Ruled non-sabotage by all involved

Contributing factors:

- Helped Identify & Resolve the Incident
- Wrote the Incident Report

Cool Beans

All your mitigations are positive validations
They don't reject unsigned stuff

Remember when *users* had to validate HTTPS?

Source: https://blog.mozilla.org/security/files/2019/10/identity_icons.png

Before:

 example.com

  https://example.com

After:

  example.com

  https://example.com

Project-Level Mitigations

When all your devs are signing

Require Signed Commits

Default	Vigilant	State
Verified	Verified	✓
Verified	Partially Verified	✓
Unverified	Unverified	✗
(Nothing)	Unverified	✗

Signature isn't enough

GitHub has an internal key you don't control!

Shai-Hulud Worm

- Harvests
 - NPM Tokens
 - GitHub PATs
 - API Keys
 - AWS
 - Azure
 - GCP
- Persistence via `shai-hulud-workflow.yml`

Controlling Authorized Keys with `sq-git`

```
$ sudo apt install sq-git
$ sq-git init
$ sq-git policy authorize --project-maintainer "Gabriel Fournier gab@devopsdays.mtl" 8F570C8B1C01FCC8
$ git add openpgp-policy.toml
$ git commit -m 'Add signing policy.'
$ git push origin signing-policy
```

sequoia-pgp/authenticate-commits

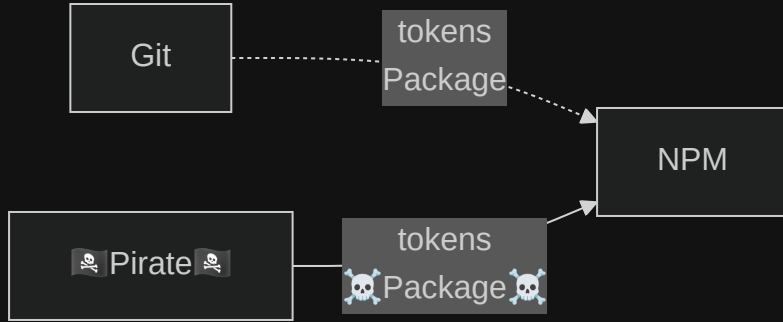
```
name: authenticate-commits
on:
  pull_request:
    types: [opened, reopened, synchronize]
jobs:
  authenticate-commits:
    runs-on: ubuntu-latest

    permissions:
      contents: read
      pull-requests: write
      issues: write

    steps:
      - name: Authenticating commits
        uses: sequoia-pgp/authenticate-commits@v1
        with:
          comment: always
```

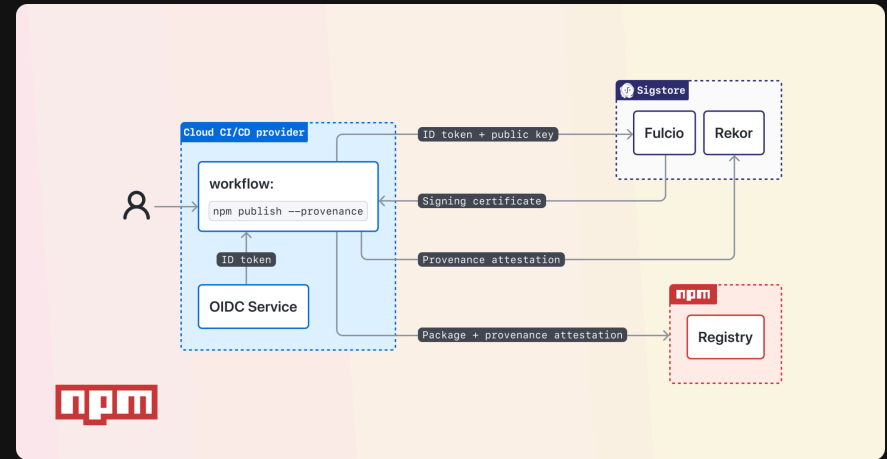
Current Supply Chain Attacks

Bypass the source git repo to publish malicious packages



OpenSSF Trusted Publishing

- Uses Short-Lived OIDC Tokens
- Allows Deactivating Traditional Tokens
- Allows MFA validations
- Limits Packages Origin to a Single Provider



Another cool part of SigStore? It offers the same flows for signing

- git commit
- container

Some predictions

Closing thoughts

I sign all my commits.

And so should you.

- Easy
- Mostly Transparent
- `git` is becoming an attack vector

Thank You!
Questions?

Powered by  Slidew

Slides: <https://github.com/carboneater/devopsdays-2025-git-signing>